



Java程序设计

Ch03-2 面向对象编程

余 烨

本章学习提示

- 本章学习Java语言中面向对象编程的具体实施过程。
  - 封装与继承
  - 覆盖与重载
  - 多态
  - 类的强制转换

目　录

1 封装与继承

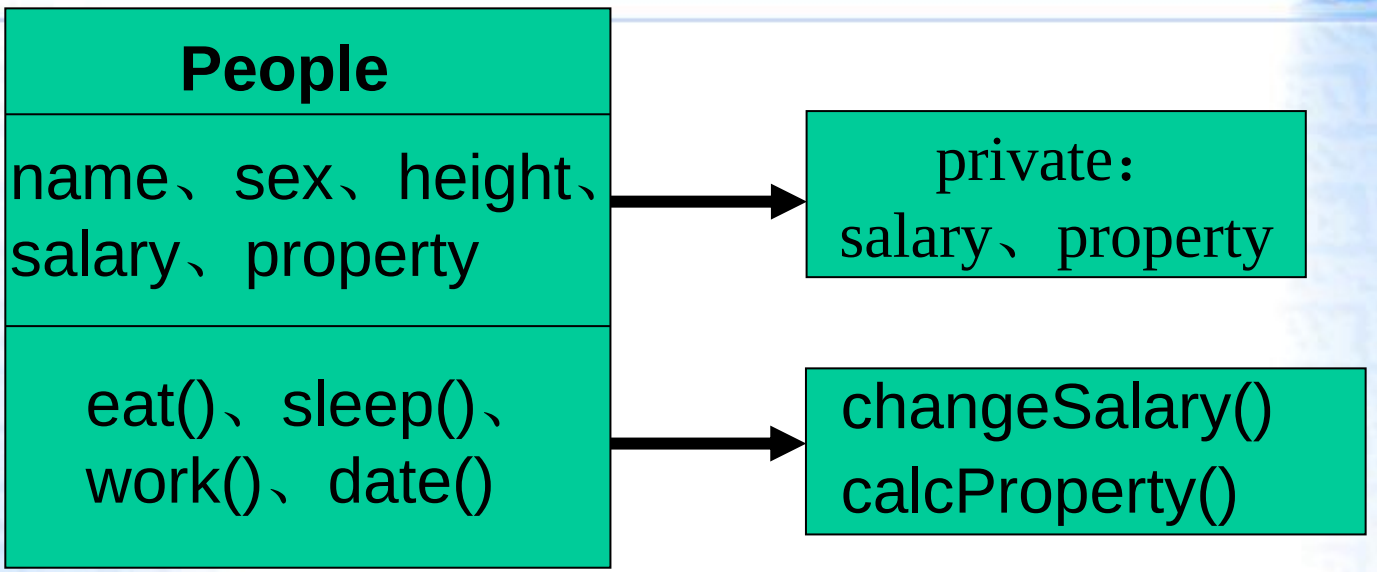
2 覆盖与重载

3 多态机制

4 类的转换

5 内部类与匿名类

封装性



将被描述的实体的属性和行为封装在一起，看作一个不可分割的整体，形成程序中不可分割的独立单位。

被封装的某些信息在封装体外是不可见的，只能通过内部的方法来改变它，通常称这些信息被隐藏了。

被封装的属性和行为中，规定了有些被隐藏在封装体内，有些外部可见的不同的访问权限。

回顾不同的访问权限

	public	protected	default	private
同类	√	√	√	√
同包不同类	√	√	√	×
不同包，子类	√	√	×	×
不同包，非子类	√	×	×	×

封装

- 所谓**封装**表现在以下几个方面
  1. 在类的定义中设置对对象中的成员变量和方法进行访问的权限。
  2. 提供一个统一供其它类引用的方法
  3. 其它对象不能直接修改本对象所拥有的属性和方法。
- 封装的作用
  - 通过封装，实现了模块化和信息隐藏，保证了数据的安全性和系统的严密性
  - 封装实现了使用者和设计者的分离，提高了程序的可重用性

类的继承——引例

- 定义一个动物的类，具有高度、重量等特征，具有吃饭、睡觉、呼吸等行为。
- 定义一个鱼类，具有同样的特征和行为。

怎么办？



要重新再写一遍么？



用继承吧！



通过关键字extends让鱼类继承动物类

Like mother, like father

实 例

```
//用extends实现类的继承
class Animal {
        protected float height, weight;
        void eat()  {  System.out.println("animal eat");  }
        void sleep()  { System.out.println("animal sleep");  }
        void breathe()  {  System.out.println("animal breathe");  }  }
class Fish extends Animal   //鱼类继承了Animal类
{    }
class Integration   {
        public static void main(String[] args)   {
                Animal an = new Animal();
                Fish fh = new Fish();
                an.breathe();
                fh.breathe();
        }    }
```

Example5_01

类的继承

- 理解继承是理解面向对象程序设计的关键
- Java中，通过关键字**extends**继承一个已有的类，被继承的类称为父类（超类，基类），新的类称为子类（派生类）。
- 在Java中，不允许多重继承。
  - class Fish { … }
  - class People {…}
  - class Mermaid extends Fish, People {…}
  - 每个类最多只有一个直接的父类



- 终止继承——关键字final

类的继承

- 类之间的继承关系是现实世界中遗传关系的直接模拟，它表示类之间的内在联系以及对属性和操作的共享，即子类可以沿用父类（被继承类）的某些特征。子类也可以具有自己独立的属性和操作。
- 子类从父类继承有两个主要的方面：
  1. **属性的继承**。例如：鱼可以继承动物的高度、重量等属性。
  2. **方法的继承**。例如：鱼类可以继承动物吃饭、睡觉、呼吸的行为。
- 属性继承和隐藏：子类不能访问父类的**private**成员，但子类可以访问其父类的**public**、**protected**成员。

构造方法的继承

- 构造方法的继承应遵守以下原则

1. 如果子类没有构造方法，则它继承父类无参数的构造方法作为自己的构造方法；（子类可以**无条件的继承父类不含参数的构造方法**）
2. 如果子类有构造方法，那么在创建子类对象时，则将先执行继承下来的父类的构造方法，然后再执行自己的构造方法

实 例

```
class Animal {
        float height, weight;
        Animal() {
                System.out.println("Animal is constructed without parameters"); }
        Animal(float height, float weight) {
                System.out.println("Animal is constructed with parameters"); }
        void eat() {
                System.out.println("animal eat"); }
        void sleep() {
                System.out.println("animal sleep"); }
        void breathe() {
                System.out.println("animal breathe"); }
}
```

实 例

```
class Fish extends Animal  {
        float length;
        Fish()  {
                super(1,1);
                System.out.println("Fish is constructed without parameters");  }
        Fish(float length)  {
                System.out.println("Fish is constructed with parameters");   }
}
class Integration
{
        public static void main(String[] args)
        {
                Fish fh = new Fish();
                Fish  fh = new Fish(3);
        }  }
```

Example5_02

目　录

1 封装与继承

2 覆盖与重载

3 多态机制

4 类的转换

5 内部类与匿名类

覆 盖

- 引例
  - 鱼的呼吸方式跟动物的呼吸方式不一样，鱼使用的是一种吐泡的方法。

怎么办？



解决方法：

重写鱼的呼吸方法！



方法的覆盖

```
class Fish extends Animal  {  //鱼类继承了Animal类
      void breathe()  //重写了动物类的breathe方法，这种机制称为覆盖
      {
            System.out.println("fish breathe: bubble");
      } }
class Integration
{
      public static void main(String[] args)
      {
            Animal an = new Animal();
            Fish fh = new Fish();
            an.breathe();
            fh.breathe();
      } }
```

实 例

Example5_03

覆盖 (Overriding)

- 在子类中定义一个与父类**同名、返回类型、参数类型均相同**的方法，称为方法的覆盖(Overriding a method definition)。

A derived class inherits methods that belong to the base class. However, if a derived class requires a different definition for an inherited method, the method may be redefined in the derived class. This is called **overriding** the method definition.

- 覆盖发生在子类与父类之间。

方法的重写规则

- 两同两小一大
  - 两同：方法名相同，形参列表相同
  - 两小：子类方法返回值类型应比父类方法返回值类型更小或相等；子类方法声明抛出的异常类型应比父类抛出的异常更小或相等。
  - 一大：子类方法的访问权限应比父类方法的访问权限大或相等。注意：覆盖方法和被覆盖方法要么都是类方法，要么都是实例方法，不能一个是类方法，一个是实例方法。

```
class BaseClass
{    public static void test()  {……}   }
class SubClass extends BaseClass
{   public void test()  {……}  }
```



提示错误：被覆盖的方法为static

覆盖 (Overriding)

- 当子类覆盖了父类方法后，子类的对象将无法访问父类中被覆盖的方法。但还可以在子类方法中调用父类中被覆盖的方法。



问题：如何在鱼类重写了父类的breathe方法后，又能调用到父类被覆盖了的breathe方法？

用super

```
class Fish extends Animal  {
    int height;
    Fish() {  System.out.println("Fish Constructed");  }
    void breathe()  {
        super.breathe();    //利用super调用父类中被隐藏的方法
        super.height = 50;  //利用super调用父类中被隐藏的变量
        super.breathe();  //利用super调用被隐藏的变量breathe()方法
        System.out.println("fish breathe: bubble");
    }  }
class Integration  {
    public static void main(String[] args)  {
        //Animal an = new Animal();
        Fish fh = new Fish();
        //an.breathe();
        fh.breathe();
    }  }
```

实 例

Example5_04

super

- 特殊变量super，提供了对父类的访问；
- 可以使用super访问父类被子类隐藏的变量或覆盖的方法；
- 每个子类构造方法的第一条语句，都是隐含地调用super()，如果父类没有这种形式的构造函数，那么在编译的时候就会报错。
- 对于父类中包含有参数的构造方法，子类可以通过在自己的构造方法中使用**super**关键字来引用，而且必须是子类构造方法中的第一条语句。

Example5_05

重载

- 所谓**方法的重载**，是在类中创建了多个方法，这些方法**具有相同的名称**，但有**不同的参数（类型不同或参数个数不同）**和**不同的实现**。
- 在调用方法时依据其参数个数及类型**自动选择相匹配的方法**去执行。达到各种对象处理类似问题时具有统一的接口目的。
- 可以定义多个构造方法，根据类实例初始化时给定的**参数列表**决定使用哪一个

```
class Point
{
    int px, int py;
    Point(int a, int b) {
        px = a; py = b; }
    Point() {    }
    void Output() {
        System.out.println( “(” + px + ”,” + py + “)” );    }
    public static void main(String[] args) {
        Point pt1 = new Point(3, 3);
        Point pt2 = new Point();
        pt1.Output();
        pt2.Output();
    } }
```

重载之例

Example：

定义一个用于计算的类，该类中至少有一个计算面积的方法calcArea，可以分别对矩形、圆、三角形等面积进行计算。

分别计算边长为5.0，半径为5.0及长、高分别为5.0、10.0的三角形的面积。

Calculator.java

目　录

1 封装与继承

2 覆盖与重载

3 多态机制

4 类的转换

5 内部类与匿名类

多态机制

- 引例

```
class Integration {
    static void fn(Animal an) {
        an.breathe();   }
    public static void main(String[] args)
    {
        Fish fh = new Fish();
        Animal an;
        an = fh;
        Integration.fn(an);
    }
}
```

Example5_06



输出什么

将一个子类对象的引用传给声明为父类的一个对象变量，如果子类有的方法就调用子类的方法，如果子类没有的方法就调用父类的方法

多态机制

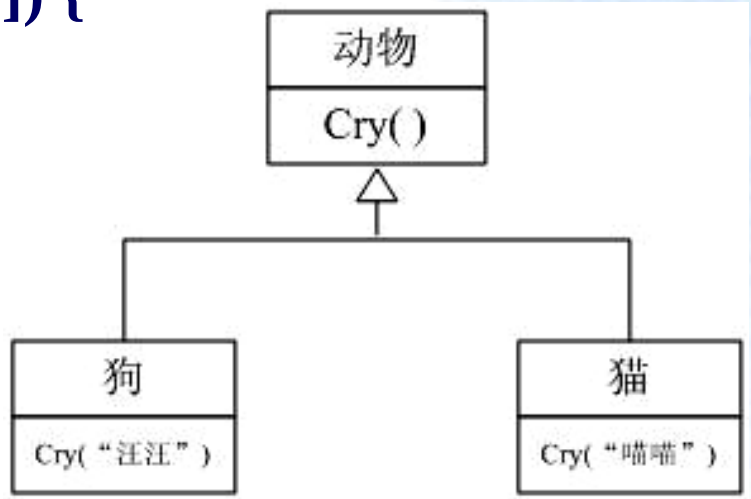
- 多态机制是面向对象程序设计的一个重要特征。
- 通过覆盖父类的方法，在运行时根据传递的对象引用，来调用相应的方法。
- **编译时多态（即：重载）**
  - 在编译时刻，根据调用方法传递参数的不同，调用相应的方法
  - 表现为**方法名相同，而参数列表不同**。
  - 典型：System.out.println(); 它的参数有很多种。
- **运行时多态**
  - 在运行时刻，根据传递的对象引用的不同，判断到底是哪个类（父类还是子类的），并调用相应对象的方法

Code - 多态

```
class 动物 {
    void cry( ) {  // do nothing  } }
class 狗 extends 动物 {
    void cry() {  System.out.println("汪汪....."); } }
class 猫 extends 动物 {
    void cry() { System.out.println("喵喵....."); } }
public class TestCry {
    public static void main(String args[]) {
        动物 dongwu;
        dongwu=new 狗();
        dongwu.cry();
        dongwu=new 猫();
        dongwu.cry();
    } }
```

Woof

Mew



“动态绑定”的思想，即：程序在执行过程中，根据不同的对象类型（是子类还是超类）自动执行相对应的方法的过程叫做动态绑定。

父类和子类中相同的成员变量

- 属性没有多态的概念，在java中只有普通的实例方法才可以实现多态。
- 成员变量不存在覆盖(override)问题，即使子类中声明了和父类完全一样的成员变量，也不会覆盖掉父类中的成员变量。而是在子类实例化时，会同时定义两个成员变量。子类可以同时访问到这两个成员变量，而父类却不能访问到子类的成员变量。
- 具体在方法中使用成员变量时，究竟使用的是父类还是子类的成员变量，由方法所在的类决定，即：方法在父类中访问和执行，访问的是父类的成员变量；方法在子类中定义（包括覆盖父类的方法）和执行，访问的是子类的成员变量。

实训一

```
class Tools
{
    Tool t1 = new Tool(1);
    static Tool t2 = new Tool(2);
    Tools()
    {
        System.out.println("Tools()");
        t4 = new Tool(44);
     }
     static Tool t3 = new Tool(3);
     Tool t4 = new Tool(4);
}
```

```
class Tool
{
    Tool(int i)
    {
        System.out.println( "Tool(" + i + ")" );
     }
}
```

```
public class TestStatic
{
     public static void main(String[] args)
     {
          System.out.println("Creating new Tools() in main()");
          new Tools();

          System.out.println("Creating new Tools() in main()");
          new Tools();
     }
}
```

TestStatic.java

在进行类的实例化时，如果是第一次进行实例化，则执行顺序为：初始化静态成员变量、 非静态成员变量、构造函数；如果不是第一次进行实例化，则执行顺序为：非静态成员变量、构造函数；

```
Creating new Tools() in main()
Tool(2)
Tool(3)
Tool(1)
Tool(4)
Tools()
Tool(44)
Creating new Tools() in main()
Tool(1)
Tool(4)
Tools()
Tool(44)
```

实训二

```
class Blocks {
    static Tool t1 = new Tool(1);
    Tool t2 = new Tool(2);
    Blocks() {
        System.out.println("父类构造函数");
    }
}
```

```
class SubBlocks extends Blocks {
    static Tool t1 = new Tool(3);
    Tool t2 = new Tool(4);
     public SubBlocks() {
        System.out.println("子类构造函数");
    }
}
```

```
class TestStaticSub
{
        public static void main(String[] args)
        {
            new SubBlocks();
            System.out.println(“One more time:”);
            new SubBlocks();
      }
}
```

TestStaticSub.java

如果实例化的类有父类存在，则：

在第一次进行子类实例化时，执行顺序为：初始化父类静态成员变量、子类静态成员变量、父类非静态成员变量、父类构造函数、子类非静态成员变量、子类构造函数；

如果不是第一次进行实例化，则执行顺序为：父类非静态成员变量、父类构造函数、子类非静态成员变量、子类构造函数；

```
Tool(1)
Tool(3)
Tool(2)
父类构造函数
Tool(4)
子类构造函数
One more time:
Tool(2)
父类构造函数
Tool(4)
子类构造函数
```

实训三

```
class Blocks {
   static  {
      System.out.println("父类静态初始化块");  }
   {   System.out.println("父类初始化块");       }
   Blocks()   {
      System.out.println("父类构造函数");  }
}
```

```
class SubBlocks extends Blocks {
   static  {
        System.out.println("子类静态初始化块"); }
   {    System.out.println("子类初始化块");     }

   public SubBlocks()    {
      System.out.println("子类构造函数");     }
}
```

```
class TestStaticBlocks
{
        public static void main(String[] args)
        {
           new SubBlocks();
           System.out.println(“One more time:”);
           new SubBlocks();
     }
}
```

TestStaticSub.java

父类、子类中静态块、非静态块的执行顺序与静态成员、非静态成员的初始化顺序相同。

父类静态初始化块
子类静态初始化块
父类初始化块
父类构造函数
子类初始化块
子类构造函数
One more time:
父类初始化块
父类构造函数
子类初始化块
子类构造函数

目　录

1 封装与继承

2 覆盖与重载

3 多态机制

4 类的转换

5 内部类与匿名类

对象状态的判断

- 由于子类对象可以作为父亲类对象使用。那么我们有时需要判断这个对象究竟属于哪个类。Java提供 **instanceof 关键字**。

```
Animal an = new Animal();
Fish fh = new Fish();
if(an instanceof Animal)
{
    System.out.println("an is Animal's instance);
}
```

结论：

父类不是子类的实例

子类是父类的实例

Example5_07

类的转换

- 类的转换是指**父类对象与子类对象之间在一定条件下可以转换**，但类对象的转换须遵守以下原则：

1. 子类对象可以转换为父类对象（向上转型——安全）；而父类对象不能转化为子类对象（向下转型——危险）
2. 处于相同层次的类对象不能进行转换

或者说：子类对象可以被看作为父类的一个对象（实例）；反之父类对象不能看作为子类的对象（实例）。

Code - 类的转换

```
// 处理父类与子类的转换 注意转换应遵循的两条原则, 例 PointTest.java
class Point2D{
        int X,  Y;
        public Point2D(){  //无参的Point2D构造方法
                X=0;                    Y=0;
                System.out.println("Point2D Constructed without Parameter!"); public
Point2D(int x, int y){ //有参的Point2D构造方法
                X=x;       Y=y;
                System.out.println("Point2D Constructed with Parameters!");}
        void offset (int a, int b){//位移Point2D
                X+=a;      Y+=b;
                System.out.println("Point2D Offset to " + this.X +"  " + this.Y);}
        void show_2d( ){
                 System.out.println("Point2D position  " + this.X +"  " + this.Y);
         }          }
```

```
public class Point3D extends Point2D {
        int X, Y, Z;          //定义3D自己的X,Y,Z
        public Point3D(int x, int y, int z){ //3D的有参构造函数
                X=x;   Y=y;   Z=z;
                System.out.println("Point3D Constructed with Parameters! “
                                + this.X  + "  " + this.Y + "  " + this.Z); }

        public Point3D(Point2D p2d,int z){ //由2D扩展的构造函数
                X=p2d.X;   Y=p2d.Y;
                Z=z;
                System.out.println("Point3D Constructed from Point2D and Z!  "
                                + this.X + "  " + this.Y + "  " + this.Z); }

        void offset(int a, int b,int c){  //3D点的位移
                this.X+=a;  this.Y+=b;  this.Z+=c;
                System.out.println("Point3D Offset!/n");  }
        void show_3d( ) {
                System.out.println("Point3D position  " + this.X +"  " + this.Y  +"  " +
this.Z );
          }        }
```

```
public class PointTest {
    public static void main(String[] args){
        Point2D p2d1, p2d2;
        Point3D p3d1, p3d2;
        p2d1 = new Point2D(2,2);    //生成父类对象
        p3d1 = new Point3D(3,4,5); //生成子类对象
        p3d1.show_2d();
        p3d1.show_3d();

        p2d2 = p3d1;                //父类引用指向子类对象
        p2d2.show_2d();             //引用父类的成员变量
        p2d2.show_3d();             //父类不能引用子类的方法

        p3d2 = (Point3D)p2d1;
        //强制将父类对象转换为子类对象，系统运行时会出现异常
    }
}
```

TestPoint

面向对象的三大特性

- 封装（Encapsulation）
- 继承（Inheritance）
- 多态（Polymorphism）

目　录

1 封装与继承

2 覆盖与重载

3 多态机制

4 类的转换

5 内部类与匿名类

内部类

- 在一个类中定义另外一个类，这个类就叫做内部类或内置类（inner class）
- 特点
  - 内部类和外部类之间建立逻辑上的联系
  - 当我们建立一个inner class时，其对象就拥有了与外部类对象之间的一种关系，这是通过一个特殊的**this** reference形成的，使得内部类对象可以随意的访问外部类中的成员
  - 外部类（outer class）控制了内部类的可见性
  - **Inner类**可以定义在一个方法内，它能够访问方法中用final定义的局部变量或参数.

内部类

- 内部类可以声明为private或protected，还可以声明为abstract或final。当一个类中嵌套另一个类时，访问保护并不妨碍内部类使用外部类的成员
- 被定义为static型的内部类将自动转化为顶层类，**不能再访问外部类中非静态的成员变量和方法**
- 非static的内部类中不能定义static型成员和变量。
- 应用内部类的**好处**：

是创建事件接收者的一个方便方法

将在GUI章中介绍

实例

```
class Outer  {
        private int index = 100;
        class Inner   {
                private int index = 50;
                void print()     {
                        int index = 30;
                        System.out.println(index);
                        System.out.println(  this.index );
                        System.out.println( Outer.this.index );    }
        void print()   {
                Inner inner = new Inner();
                inner.print();     }
        Inner getInner()    {
                return new Inner();     }
}
```

```
class Test  {
        public static void main(String[] args)
        {
                Outer outer = new Outer();
                outer.print();     //调用外部类中的print()方法
                //在外面访问内部类的两种方式
                Outer.Inner inner1 = outer.getInner();
                Outer.Inner inner2 = outer.new Inner();
                inner1.print();   //调用内部类的print()方法
                inner2.print();
        }
}
```

匿名类

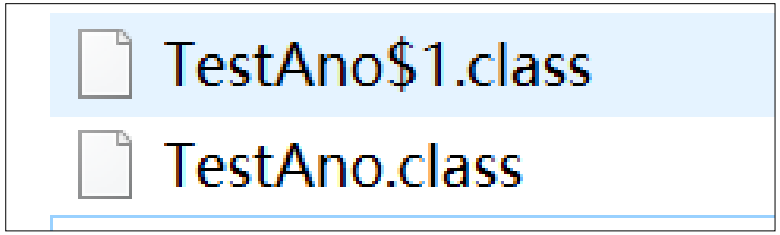
- 匿名类(Anonymous class)，就是**没有名称的类**，其**名称由Java编译器给出**，一般是形如：**外部类名称+$+匿名类顺序**，没有名称也就是**其他地方就不能引用，不能实例化，只用一次，当然也就不能有构造器**。
- 匿名类就是利用父类的构造函数和自身类体构造成一个类。

格式：new 父类() { 子类内容}

如：new Bank() {匿名类的类体}

- “父类”指子类需要继承或者实现的外部的类或接口
- 匿名类可以继承、重写父类的方法
- 匿名类一定是内部类，可以访问外部类的成员变量和方法
- 匿名类的类体中不可以声明static的成员变量和方法

```
abstract class Outer {
    public abstract void show();  }
class TestAno  {
    public static void main(String[] args)    {
            //把匿名子类的对象赋值给了父类的变量
          Outer out = new Outer()  {
                public void show() {
                    System.out.println ("run in Anonymous");
                } //匿名子类里实现了父类中的抽象方法show
          };
          out.show();
    }
```



TestAno.java

本章总结

- 本章学习了Java面向对象编程的具体实施过程。
  - 封装与继承
  - 覆盖与重载
  - 多态机制
  - 类的强制转换

Class is Dismissed!

